COMPARATIVE ANALYSIS OF MODERN TABULAR ML For

Network Intrusion Detection

Evaluating gradient boosting models for detecting cyber attacks in network traffic.

CSE-CIC-IDS2018
Main dataset

3 Models + SMOTE
Models compared

<15 min
Talk target

Farman H. Sayem · Jace C. Rea · Kunle S. Oguntoye
Group 5

Project overview: what we set out to build

Core purpose

A reproducible evaluation framework for network intrusion detection models.

- Classify network flows as benign or attack.
- Compare LightGBM, XGBoost, CatBoost, and SMOTE variant.
- Evaluate performance, probability reliability, and drift robustness.

What makes this more than a basic model comparison?

- Uses metrics that matter for cybersecurity, not just accuracy.
- Tests calibration because risk scores must be trustworthy.
- Tests drift because traffic changes across time and datasets.

Takeaway: The project is about making model evaluation more realistic for security use.

Motivation and relevance: why accuracy is not enough

Intrusion detection is a high-cost imbalance problem.

Missed attack

A false negative can mean an intrusion slips through.

False alarm

A false positive adds analyst workload and alert fatigue.

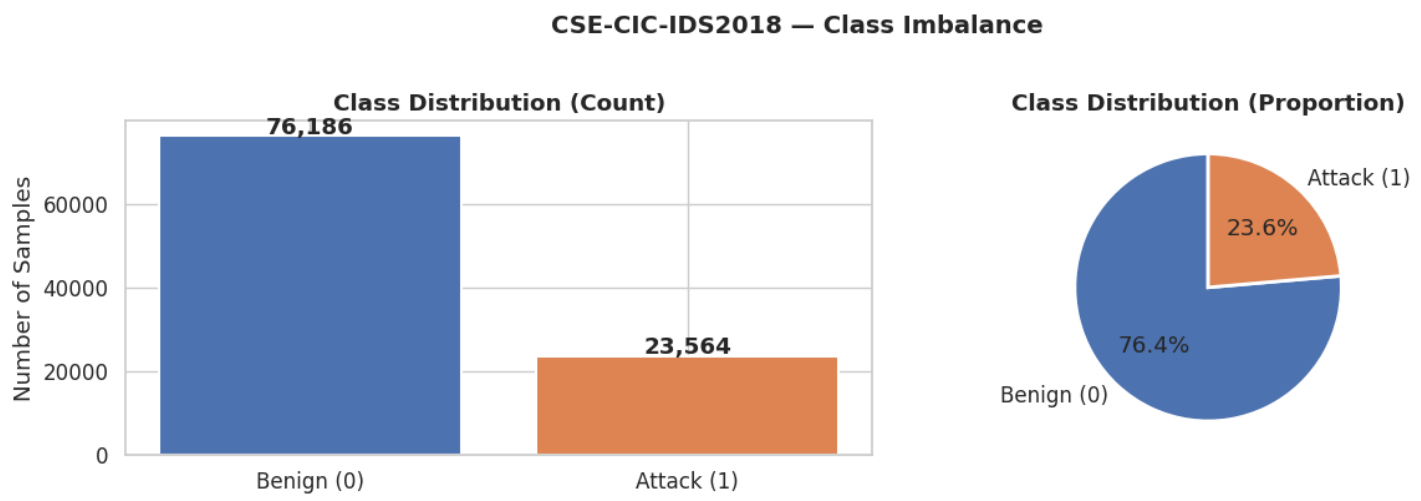
Risk score

A probability only helps if it is calibrated and actionable.

Metrics used: recall, precision, F1, PR-AUC, FPR, Brier score, and drift performance.

Takeaway: A useful detector has to catch attacks, avoid alert fatigue, and produce reliable risk scores.

Dataset overview: class imbalance in the final 2018 sample



Final data used

- Primary dataset: CSE-CIC-IDS2018.
- Drift dataset: CIC-IDS2017.
- 2018 cleaned set: 99,750 rows.
- 2017 cleaned set: 48,902 rows.

Why it matters

The class split is not extreme, but attacks are still the minority class. PR-AUC and recall are more informative than accuracy.

Takeaway: The model must learn attack behavior from a smaller attack class, so imbalance-aware evaluation matters.

Project setup and Colab code stack

Environment

- Google Colab + Google Drive-mounted datasets.
- Fixed random seed: 42 for reproducibility.
- Large CSVs streamed and sampled to stay within Colab memory.

Python libraries

- pandas / NumPy for data handling.
- scikit-learn for splitting, pipelines, metrics, calibration.
- imbalanced-learn for SMOTE.
- LightGBM, XGBoost, CatBoost for model training.

Challenges with real data

- Streamed multi-GB CSVs in chunks to avoid crashes.
- Stripped inconsistent column whitespace and bad rows.
- Binarized labels: Benign = 0, all attacks = 1.
- Aligned only shared usable features across 2018 and 2017.

Pipeline layout

Load → Clean → Align features → Split → Train → Evaluate → Calibrate → Threshold → Drift test

Design goal: every transformation is reusable and fitted only where appropriate to avoid leakage.

Takeaway: The notebook was built as a reproducible pipeline for a reliable system rather than a one-off model script.

Preprocessing and split design

Cleaning choices

- Replaced infinite values with missing values.
- Removed duplicate rows.
- Dropped identifiers like IPs, ports, timestamps, and flow IDs.
- Clipped extreme outliers at the 99.9th percentile.

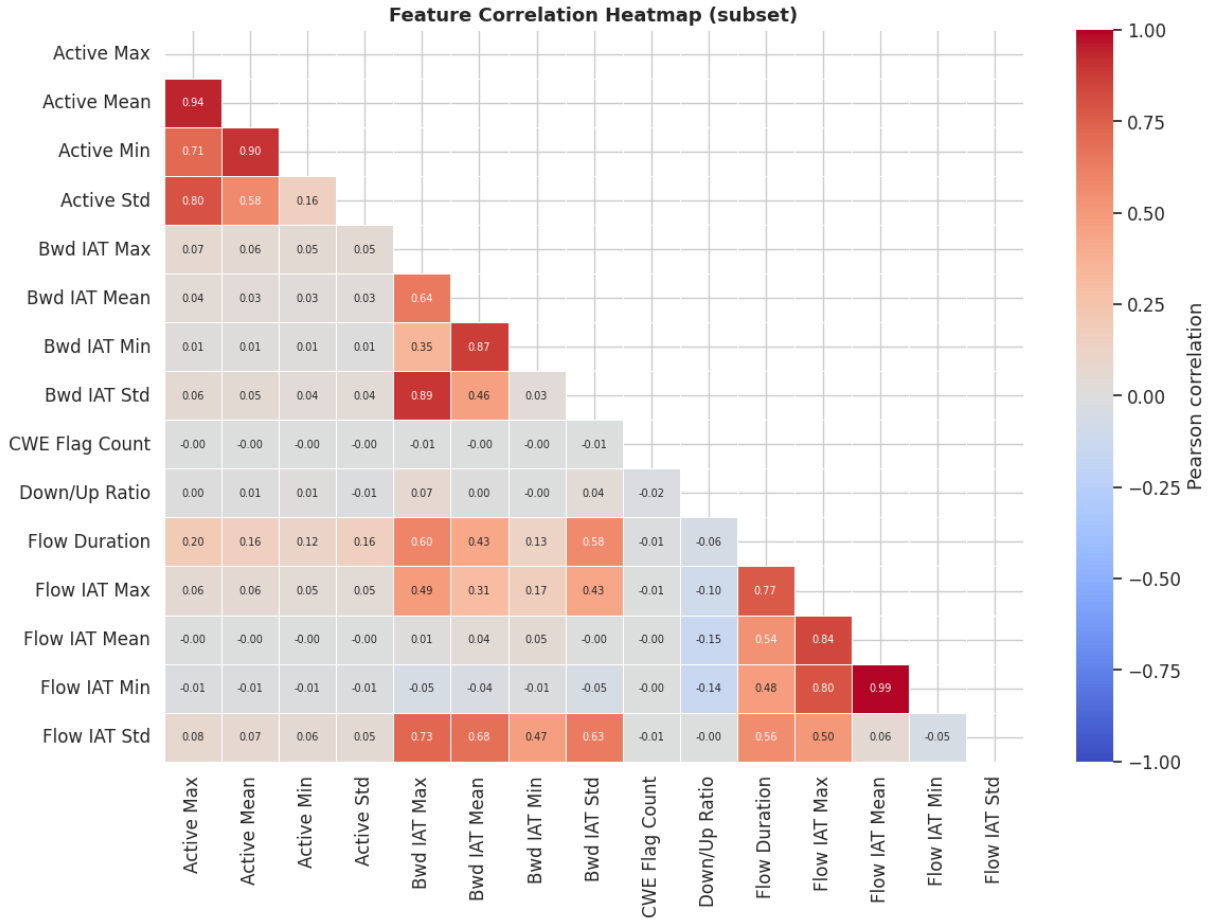
Final modeling setup

- 23 shared numeric features after 2018/2017 alignment.
- 70/15/15 stratified train/validation/test split.
- Preprocessor fit only on training data.
- 2017 drift set transformed with the 2018-fitted preprocessor.

69,824	14,963	14,963	23	~23.6%
Train rows	Validation rows	Test rows	Features	Attack rate

Takeaway: The split and preprocessing choices were designed to reduce leakage and keep comparisons fair.

EDA: timing features were strongly related



What to notice

- Several IAT timing features are highly correlated.
- Flow IAT Min and Flow IAT Mean are nearly redundant.
- Active Mean and Active Max also move together strongly.

Why it matters

The model is likely learning groups of related timing behavior rather than one isolated feature at a time.

Takeaway: The EDA suggested timing behavior would be a major signal for intrusion detection.

Model design: baseline comparison plus evaluation layers

Baseline model comparison

- LightGBM: fast gradient boosting for tabular data.
- XGBoost: strong regularized boosting baseline.
- CatBoost: robust out-of-box boosting comparison.
- LightGBM + SMOTE: oversampling test against class weighting.

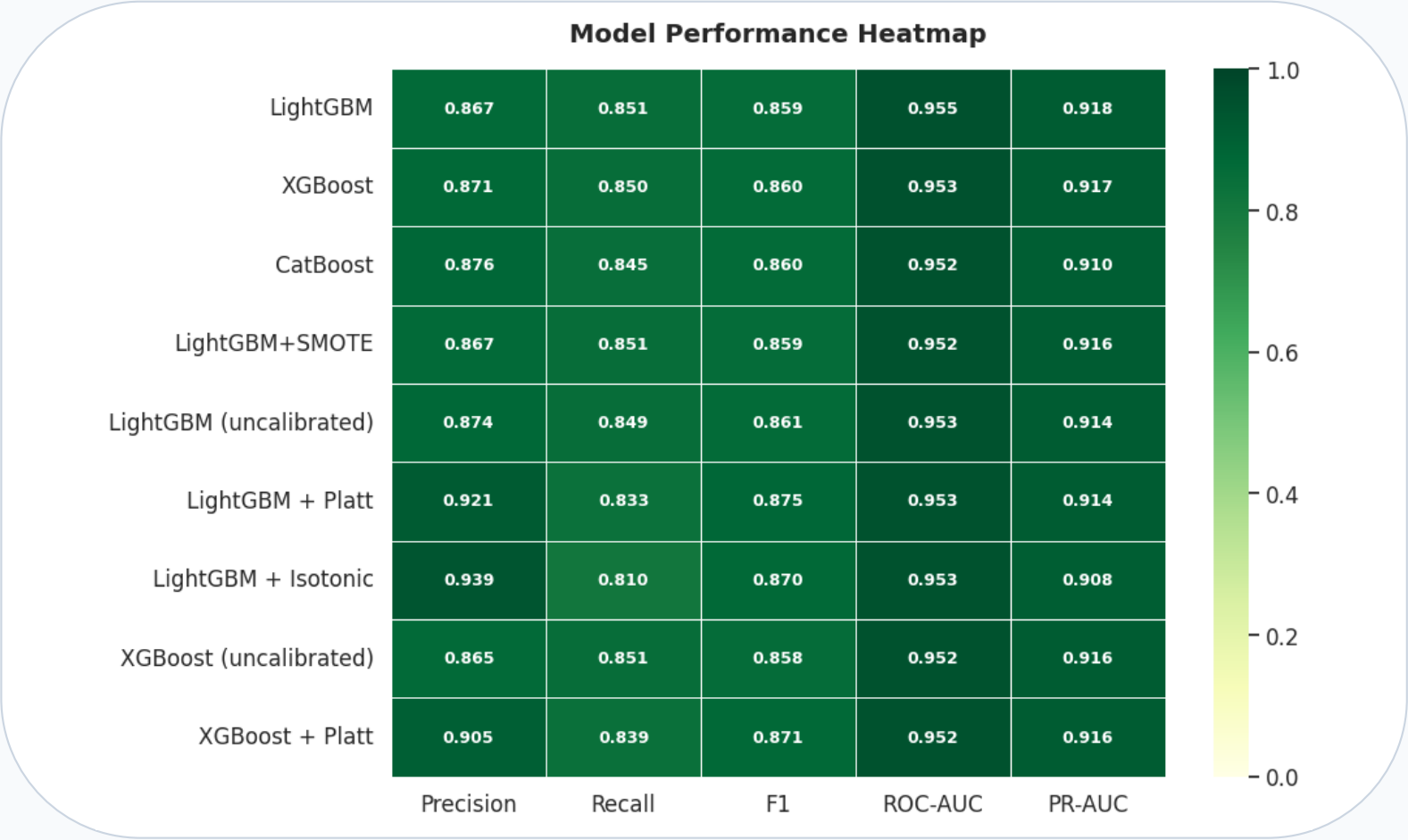
What we added beyond baseline

- Class weighting to penalize missed attacks more heavily.
- RandomizedSearchCV attempted with PR-AUC objective.
- Calibration with Platt and Isotonic methods.
- Threshold sweep and cross-year drift testing.

Because the tuning cell reported nan CV PR-AUC, avoid overclaiming “tuning improved the model.” Treat the final models as final evaluated / search-selected versions.

Takeaway: The project compared baseline models, then tested the choices that matter in real intrusion detection.

Results: overall model performance on 2018 test data

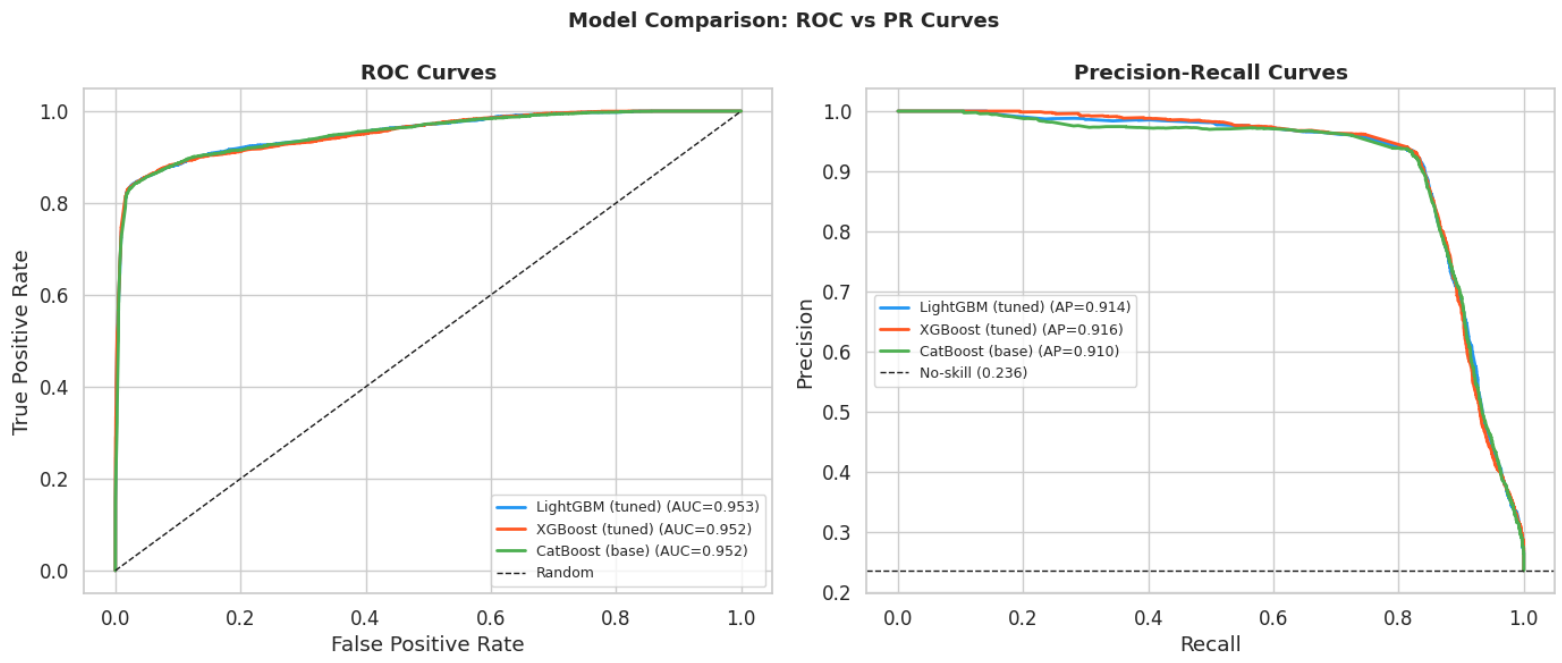


Main comparison

- LightGBM base: best PR-AUC at 0.918 and recall at 0.851.
- XGBoost was nearly tied: PR-AUC 0.917 and recall 0.850.
- CatBoost was competitive but slightly lower PR-AUC.
- SMOTE did not clearly outperform class weighting.

Takeaway: LightGBM was the strongest same-year base model, but XGBoost was extremely close.

Results: ROC and PR curves show strong but similar ranking



How to read this

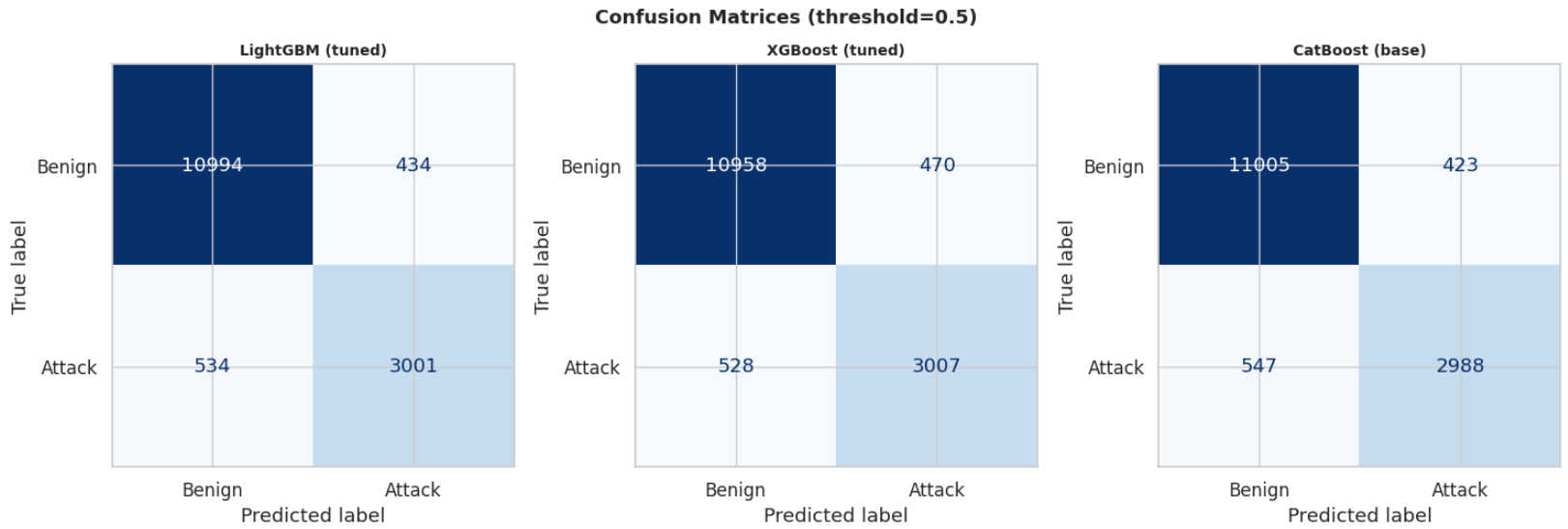
- ROC-AUC is high for all models, around 0.952–0.953.
- PR-AUC is more important because attacks are the minority class.
- Curves are close, so model choice alone is not the full story.

Why PR-AUC matters

It focuses on how well the model finds attacks while controlling false alarms.

Takeaway: The models rank attacks well on 2018 data, but calibration and thresholds determine practical usefulness.

Results: operational impact at the default 0.5 threshold



At threshold = 0.5

534
LightGBM missed attacks

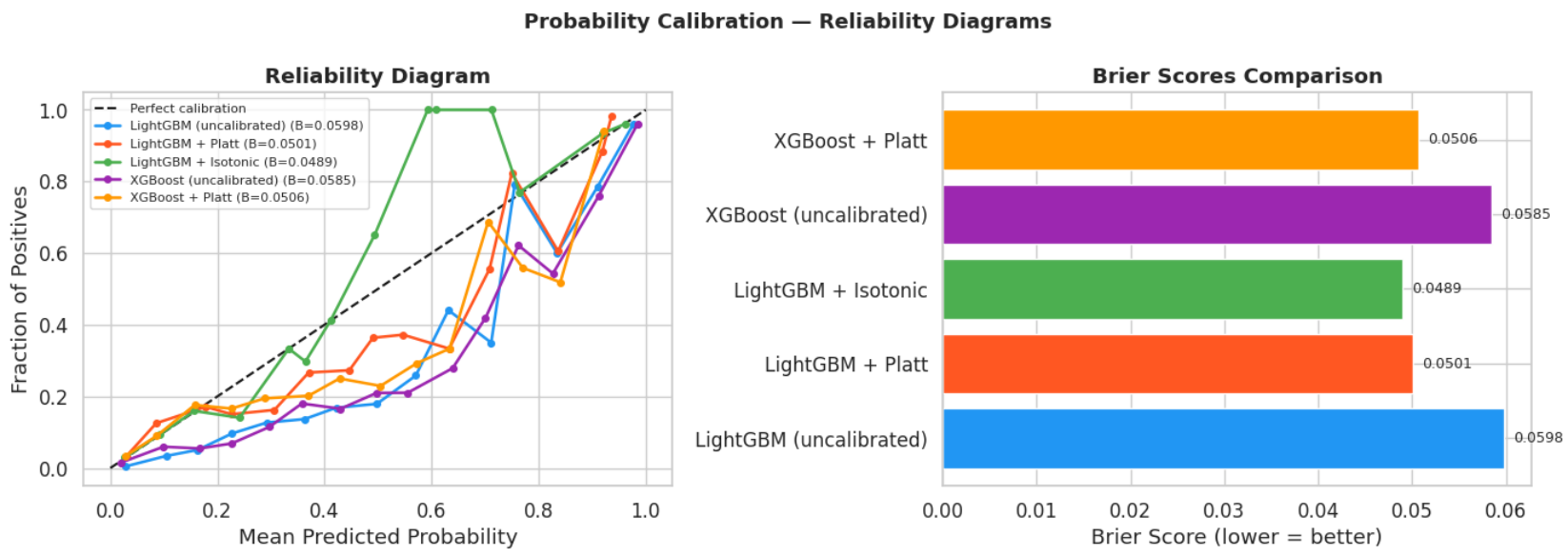
528
XGBoost missed attacks

547
CatBoost missed attacks

XGBoost caught slightly more attacks; CatBoost created slightly fewer false positives.

Takeaway: Confusion matrices translate metrics into missed attacks and analyst workload.

Results: calibration made risk scores more reliable



What changed?

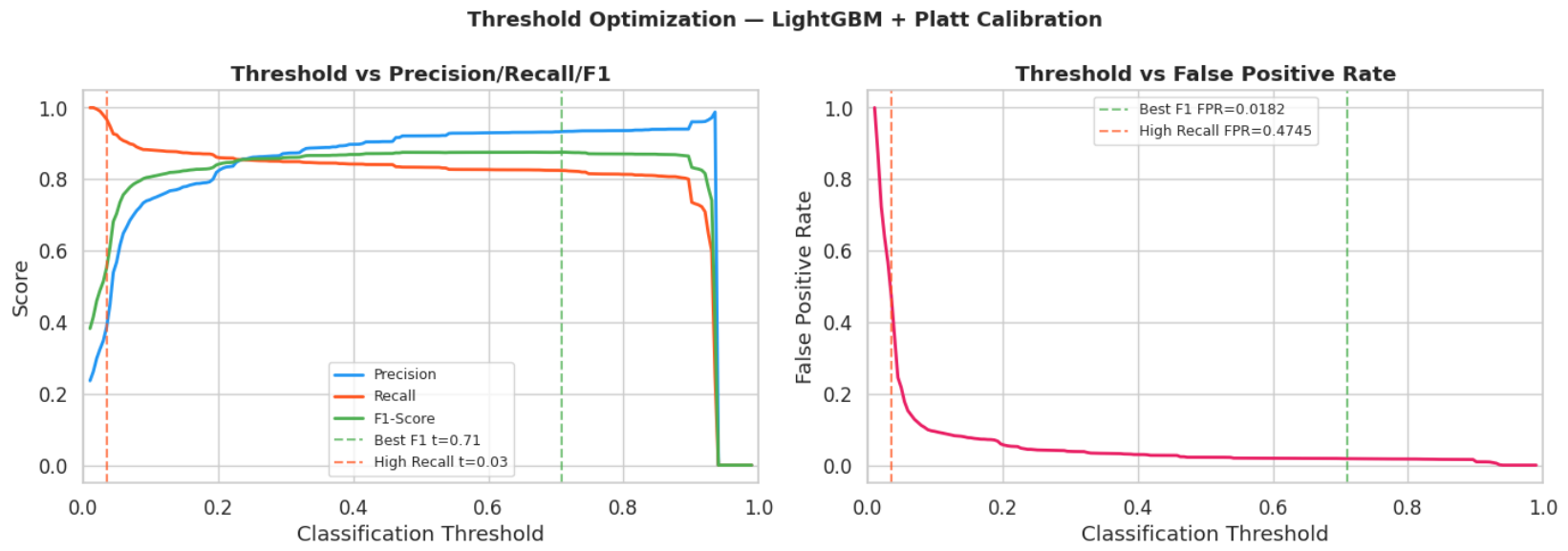
- Brier score measures probability error; lower is better.
- LightGBM + Isotonic had the best Brier score: 0.0489.
- Calibration improved risk-score reliability.

Tradeoff

Calibration can make the model more conservative at the same 0.5 threshold, which may reduce recall.

Takeaway: Calibration helps when probabilities are used for risk scoring, not just yes/no classification.

Results: threshold tuning controls the recall vs false-alarm tradeoff



Two operating points

Max F1

t ≈ 0.709
F1 = 0.8755
Precision = 0.9334
Recall = 0.8243

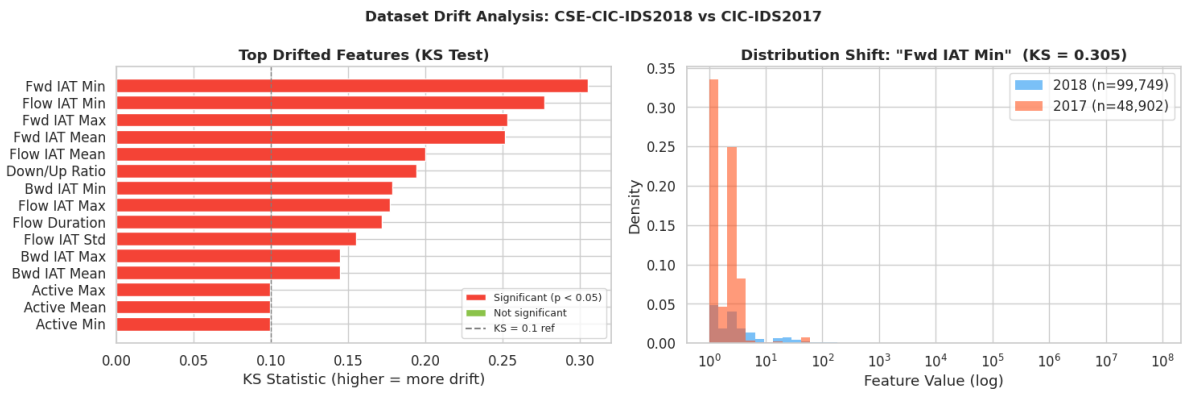
High recall

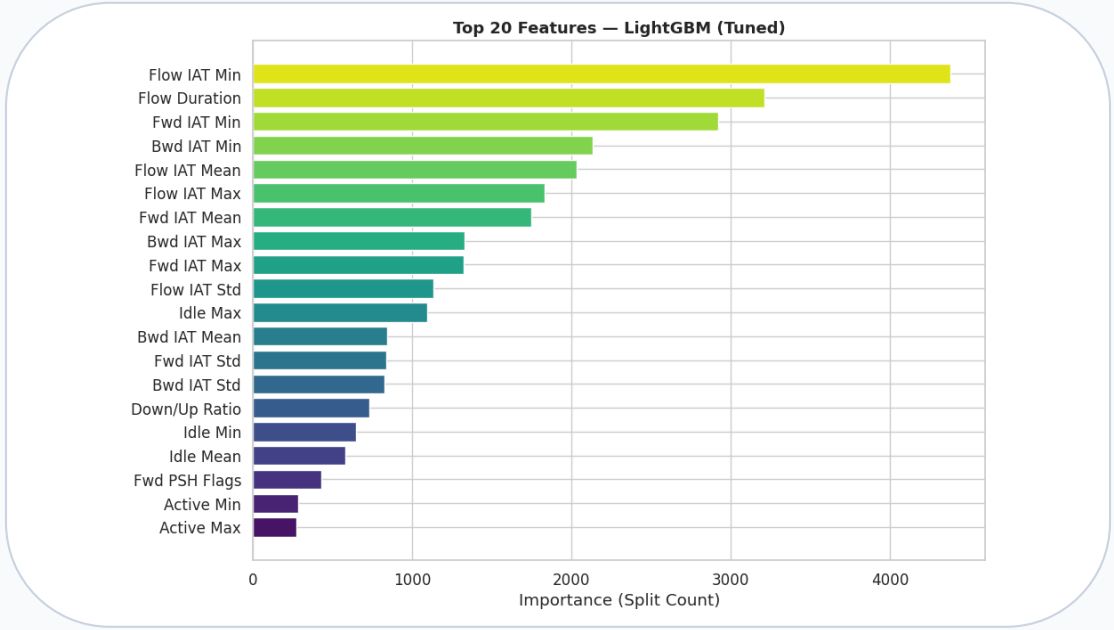
t ≈ 0.035
Recall = 0.9675
FPR = 0.4745

Interpretation: high recall catches more attacks, but creates far more false alarms.

Takeaway: The best threshold depends on risk tolerance and analyst capacity.

Results: drift exposed the biggest weakness





Connection

The model relied heavily on IAT/timing features. Those same features shifted most between 2018 and 2017, which helps explain the severe cross-year drop.

PR-AUC under drift

LightGBM: 0.914 → 0.260
XGBoost: 0.916 → 0.335
CatBoost: 0.910 → 0.255

Takeaway: Strong same-year results did not guarantee cross-year robustness; XGBoost degraded least, but all models degraded.

Challenges, limitations, and future work

Challenges / limitations

- Dataset size required sampling and chunked loading.
- 2018 and 2017 feature formats did not align perfectly.
- Binary labels hide differences between attack types.
- Random split is less realistic than a time-based deployment split.
- Tuning output had nan CV PR-AUC, so tuning claims should be cautious.

Future work

- Fix and rerun hyperparameter tuning.
- Test temporal train/test splits (train on earlier dates, test on later)
- Expand from binary to multi-class attack classification.
- Add live drift monitoring and retraining triggers.
- Evaluate thresholds based on analyst workload constraints.

Final answer

LightGBM was the best same-year speed/performance choice; XGBoost was most robust under drift; calibration and threshold tuning are essential before operational use.

Takeaway: Practical intrusion detection needs model selection, calibration, thresholding, and drift monitoring together.

Any Questions?